Milestone 6, Part 2: Hand-Strength Tables and Card Buckets

holdem-cfr walkthrough

What a bucket is for

The blueprint will be trained with Monte Carlo CFR on an *abstract* game. In that game a player does not see their exact cards. They see a **bucket**, a small integer that groups together hands that play alike. Regrets and strategies are stored per (betting history, bucket), so the bucket count controls the blueprint's size. With 50 buckets per postflop street, every flop betting sequence has 50 rows of regrets instead of 1.3 million.

This abstraction throws information away, so the goal is to throw away the least useful part. The standard features are **hand strength** and its expected square, and computing them for every class exactly is the first half of this part.

Hand strength, exactly, for 123 million hands

On the river, a hand's strength (HS) is the probability of beating a uniformly random opponent hand, counting ties as half. Of the 52 cards, 7 are visible (2 hole + 5 board), leaving 45 unseen cards and therefore C(45, 2) = 990 possible opponent hands. HS = (wins + ties/2) / 990.

Evaluating 990 opponents for each of the 123 million river classes would take about 120 billion evaluator calls. Two ideas from earlier milestones bring this down to minutes.

Boards up to isomorphism. The isomorphism indexer works on boards alone too. There are only 134,459 distinct river boards up to suit relabelling (and 1,755 flops, 16,432 turns, numbers the tests pin). Every river class has a member whose board is one of these representatives, so it is enough to process the representative boards.

One sort per board. On a fixed board, rank all 1,081 possible hands once with the evaluator and sort them. Then one ascending sweep gives every hand its win count. This is exactly the showdown trick from the postflop solver: keep a running total of hands already passed, plus a per-card running total, and subtract the hands that share a card with the current one, because they cannot be the opponent's hand. Ties are handled by processing hands of equal rank as a group. For the tie count, start from the group size, add 1 to cancel the hand itself (it shares both of its own cards, so it is subtracted twice), and subtract the group hands that share each card.

The tie calculation produced the one real bug of this part. Written in the obvious order, `group - shared_a - shared_b + 1`, it underflowed an unsigned integer in the middle of the expression, even though the final answer is never negative. Rust panics on integer overflow in debug and test builds instead of silently wrapping, so the first test run caught it immediately. Reordering to `group + 1 - shared_a - shared_b` fixes it. The test compares the sweep against brute force over all 990 opponents on boards chosen to stress ties: a full house on the board, a royal flush on the board (where every hand ties and HS is exactly 0.5), and a board straight.

Earlier streets reuse later ones. On the turn, a hand's E[HS] and E[HS²] are averages of the river table over the 46 cards that can come. The flop then averages the *turn* table over the 47 turns, because of the law of total expectation: the expected river value averaged over turn cards is the expected value over turn-and-river runouts. Preflop averages the flop table over all 19,600 flops. No street ever touches the 990 opponents again. The river is the only street that runs the evaluator.

Why E[HS²]

Compare a flush draw and a weak made hand that both win about 40% of the time on the flop. Their E[HS] is the same, but they play very differently. The draw usually ends up either nearly unbeatable or nearly worthless, while the weak pair stays mediocre. E[HS²] separates them, because squaring rewards variance: for the same mean, the hand whose strength is spread toward 0 and 1 has the larger E[HS²]. Johanson's thesis showed this simple change makes better abstractions than E[HS]. Richer features exist, such as histograms clustered with earth mover's distance. They are an upgrade path, not a requirement.

Percentile buckets

A bucket is a range of the metric: E[HS²] on the flop and turn, HS on the river. The thresholds are **quantiles over random deals**, not over isomorphism classes. Classes are not equally likely: a class containing a suited hand on a rainbow board stands for more raw deals than some symmetric configurations do. So the builder samples two million random (hole, board) deals per street, sorts their metric values and puts thresholds at evenly spaced quantiles. Every bucket is then reached about equally often in actual play.

Preflop keeps all 169 classes. It costs nothing, and preflop mistakes are expensive.

The abstraction uses **imperfect recall**. A hand's turn bucket does not remember its flop bucket. Waugh et al. (2009) and Lanctot et al. (2012) show this is a much better use of memory than perfect-recall buckets, at the price of weaker theoretical guarantees for CFR, which is a trade every competitive bot makes.

Validation against known numbers

Three layers of tests cover this part:

- **Board counts:** exactly 1,755, 16,432 and 134,459 isomorphism classes of 3-, 4- and 5-card boards. - **The river sweep against brute force:** every sampled hand on four adversarial boards matches the 990-opponent enumeration to within 10^{-6}. - **Preflop equity against the published values.** These are the sanity anchor for the whole pipeline, because a mistake anywhere in it (river sweep, isomorphism indexing, any of the averaging steps) would show up here.

Published preflop all-in equities against a random hand are textbook figures. Ours come from tables built by sweeping 134,459 river boards and averaging upward. They match to every printed digit:

Hand	E[HS] (ours)	Published	E[HS²]
AA	0.8520	85.20%	0.7372
KK	0.8240	82.40%	0.6922
QQ	0.7993	79.93%	0.6534
AKs	0.6704	67.04%	0.5111
AKo	0.6532	65.32%	0.4862
T9s	0.5403	54.03%	0.3906
72o	0.3458	34.58%	0.2073
32o	0.3230	32.30%	0.1995

The E[HS²] column shows why it is the better feature. T9s has much less raw equity than AKo (0.54 against 0.65), but on later streets its strength is more spread out, toward straights and flushes or nothing, and E[HS²] gives that spread weight.

An opt-in test, `cargo test --release -p hunl --test strength -- --ignored`, rebuilds every table and checks three things:

- The equities above, to within 0.0006. - Suit-swapped hands get identical values. - Jensen's inequality, $E[HS^2] \geq E[HS]^2$, holds for every one of the 1.3 million flop classes.

It then checks the buckets. On 200,000 fresh random deals per street, every bucket is reached at a rate within a factor of two of the ideal 2%. Ties on the river make a few buckets lumpy, because thousands of hands share exactly the same strength.

Cost

On the 2-core development VM, the whole pipeline takes under four minutes:

Stage	Work	Time (cumulative)
River	134,459 boards × (1,081 evaluations + sort + sweep)	35 s
Turn	13,960,050 classes × 46 river lookups	203 s
Flop	1,286,792 classes × 47 turn lookups	221 s
Preflop	169 classes × 19,600 flops	222 s
Buckets	2M sampled deals per street, quantiles, tables	228 s

The turn dominates. It does about 640 million isomorphism-index computations, and each one includes a hash-map lookup of the suit configuration. That lookup is the obvious next optimization if this ever needs to be faster, for example by replacing the map with a perfect table over sorted shapes. The output is a 139 MB bucket file (one byte per class on every street), written to `data/` and kept out of git.

The sampled solver is ready

The training algorithm for part 3 is already written and validated. External-sampling MCCFR samples every card up front, then walks the abstract betting tree once per player. It explores every action at the traverser's nodes and samples one action from the current strategy at the opponent's nodes. Regrets live in flat arrays indexed by (betting node, bucket). Many iterations run at once on rayon threads, updating those arrays with lock-free atomic compare-and-swap adds.

The test runs exactly this code on Leduc, expressed as a "sampled game" in which a player's bucket is their exact cards, and measures exploitability with the exact best response from milestone 1. Exploitability falls from 0.23 after 10,000 iterations to 0.063 after 100,000 and 0.034 after 400,000, about a second of computation. The game value approaches the published −0.0856.

Full-width CFR+ is far faster *on Leduc*: it reached 7×10^{-6} in a second and a half. Sampling only pays off in games whose trees are too large to walk in full every iteration, which is exactly the situation of the no-limit blueprint. A second test checks that single-threaded runs with a fixed seed are bit-reproducible.

Next

Part 3 trains the blueprint. External-sampling Monte Carlo CFR runs over the abstract game: 169 / 50 / 50 / 50 buckets and a small bet-size menu per street, multithreaded with lock-free atomic regret updates. To make sure the sampled algorithm is right, the identical code first runs on Leduc, where its exploitability

must track full CFR's.